to run more frequently—at the cost of a bit more CPU. This flag is enabled on every single production system I've worked on, and it has never caused problems for me—mainly because frequent garbage collection is fast and handled on a per-process basis.

There's another option to prevent memory bloat. You can put individual processes into a hibernation state. A hibernating process has its memory reduced as much as possible. But when the process receives a message, it will incur a cost to exit the hibernation state and handle the message. GenServer[4] has a `hibernate_after` option that will automatically enter hibernation when the GenServer is idle.

Both techniques are important to know about, but you likely won't need to use them for some time. Frameworks like Phoenix use hibernation with sane defaults so that you often don't need to think about it.

Now that you have the basics of processes down, let's take a look at how Elixir makes them easy with GenServer.

Go Parallel with GenServers

Processes are the lowest-level concurrency primitives in Elixir, but it's pretty rare to use them directly. Instead, you'll use libraries that let you build processes without worrying about the details. GenServer stands for "generic server," and it's the most common process library.

GenServer is part of OTP, so it's actually written in Erlang. Elixir provides the `GenServer` module to seamlessly bring this Erlang library into Elixir. It's a very important library, so it has been integrated into the language very well.

GenServer solves a few problems that you have already seen:

State Management
: A GenServer stores state and makes it available to all message handlers. This state can be modified in response to a message.

Seamless Messaging APIs
: GenServer provides functions to synchronously `call` into the process or to asynchronously `cast` into the process. These functions are built using `send` and `receive`. Plus, they handle cumbersome details like timeout, responses, and so on.

4. https://hexdocs.pm/elixir/GenServer.html